perplexity

1. Demo Video Script: Adjustments & Improvements

Timestamp	Original Plan	Recommended Correction / Refinement	Why This Wins Judge Points
0:00 – 0:15	Terminal + voiceover on cognitive	Add on-screen title text:	Instantly clarifies the target user
debt.	"UnderstandGuard – for teams using	and signals a serious submission to	
	AI coding agents" and briefly	skimming judges.	
	mention the hackathon name.		
0:15 – 0:40	Paste a realistic Git diff with	Keep diff small (≤ 40–50 lines).	Easy to read on small laptop screens
race condition.	Show filename and brief context	and prevents visual clutter.	
	comment (e.g., // auth.ts -		
	refreshToken flow).		
0:40 – 1:10	Reverse interview: 3 sharp	Ensure each question cites specific	Shows technical depth instead of
questions.	line numbers/functions (e.g., "Line	rushing through generic prompts.	
	52 – refreshToken()"). Focus on Q1		
	& Q2 in detail; summarize Q3.		
1:10 – 1:25	Developer types vague answer to Q2.	Add on-screen visual highlight on	Visually proves the core value: the
	the weak phrase ("handles errors	system detects shallow	
	gracefully") + small "⚠ Vague	understanding, not just syntax	
	Explanation" badge.	errors.	
1:25 – 1:40	Risk flag + edge-case test.	Explicitly voiceover: "Because Q2	Directly connects the test
	was vague, UnderstandGuard	generation to the developer's blind	
	generates this targeted adversarial	spot (differentiates from generic	
	test for refreshToken()." Show	test generators).	

Timestamp	Original Plan	Recommended Correction / Refinement	Why This Wins Judge Points
	input → failure error.		
1:40 – 1:55	Export screen: COGNITIVE_MAP.md +	Show two concrete deliverables: 1.	Delivers tangible, practical repo
summary.	COGNITIVE_MAP.md snippet (Mermaid	artifacts judges want to see.	
	diagram).2. Verdict badge: "High		
	Cognitive Debt – Review Required".		
1:55 – 2:00	Final punchline.	Conclude with impact: "For teams,	Ties the technical innovation
	that means fewer surprise	directly to business impact (a key	
	production bugs and faster, safer	rubric item).	
	code reviews."		

---

2. Tech Stack Refinements

Layer	Recommended Choice	Refinement from PDF	Rationale
Frontend	Next.js / Vite + Tailwind	Dark-mode, terminal/dashboard aesthetic.	Enables one-click flawless walkthroughs
	Include a dedicated /demo route pre-	during video recording.	
	loaded with sample diffs.		
Backend	Node.js / TypeScript	Clean single API route: POST	Keeps codebase modular and demonstrates
	/api/analyze-diff returning { questions,	a clean API contract.	
	score, riskFlags, testSnippet,		
	cognitiveMap }.		
Diff Parsing	npm library (parse-diff)	Use an in-app parser rather than	More reliable across environments and
	shelling out to raw git diff CLI.	hosted deployments.	
AI Engine	Gemini API	Add an instant fallback/mock mode with	Prevents demo failure if the API is slow
	pre-baked questions and tests.	or rate-limited during screen recording.	

---

3. Planning Artifacts to Create (Devpost Rubric Requirement)

The PDF emphasizes creating three distinct planning files in the repo rather than just one:

1. scope.md: Clear boundaries, single-task PoC definition, and explicit out-of-scope items.
2. prd.md: User personas (developers using AI agents), user stories, and measurable success criteria.
3. spec.md: Technical architecture, API contracts (/api/analyze-diff), diff parsing logic, scoring rubric, and test generation
   algorithm.
   ———————

4. Strategic Next Steps

Step	Action Item	Priority
1	Initialize the 3 planning documents: scope.md, prd.md, and spec.md.	Immediate
2	Prepare sample demo data: a realistic 40-line auth.ts diff with an unhandled 429/race condition, plus	Day 1
pre-written vague & good developer answers.		
3	Pitch at Cup of Joe Office Hours (Today, 3:00 PM ET on Discord voice) with a 1-minute elevator pitch to	Today 3 PM ET
get Joe's feedback.		
4	Build the Next.js/Vite prototype with the /demo quick-loader route.	

,,, is it corrected ?

Yes—your plan is now correctly aligned with what strong Devpost submissions look like. The script refinements, stack choices, and planning docs all match typical judging criteria (impact, innovation, execution, design, presentation) and the “one focused PoC” expectation. [1] [2] [3] [4]

Below are small but important corrections and additions to make it even tighter.

1. Demo video script: final corrections

Your table is solid; just refine a few details so it’s realistic and judge-friendly.

Timestamp	Your current plan	Small correction / addition	Why it matters
0:00–0:15	Title text: “UnderstandGuard – for teams using AI coding agents” + hackathon name.	Add **one-sentence problem** on screen, e.g.: “AI writes code. Humans must understand it.”	Judges who watch without sound still get the core idea. [1] [2]
0:15–0:40	≤ 40–50 line diff, filename + context.	Make sure the **bug is visible** in the diff (e.g., missing `await`, no retry on 429, shared mutable state).	Judges should be able to spot “something risky” even before your tool flags it. [3] [4]

Timestamp	Your current plan	Small correction / addition	Why it matters
0:40–1:10	3 questions, focus on Q1 & Q2, summarize Q3.	In the video, you can **show Q1 and Q2 fully**, then cut to Q3 as: "A third question probes X; we'll skip to the result to save time."	Keeps you under 2 minutes while still showing depth. [2] [4]
1:10–1:25	Vague answer + "⚠ Vague Explanation" badge.	Also show a **tiny score update**, e.g., "Comprehension: 2/3 → Medium Risk".	Makes the scoring feel concrete, not magical. [3]
1:25–1:40	Targeted adversarial test for `refreshToken()`.	Show **one clear assertion failure**, e.g.: `expect(status).toBe(200) // got 500`. Avoid long logs.	Judges see cause→effect quickly; long logs kill pacing. [2] [4]
1:40–1:55	COGNITIVE_MAP.md + verdict badge.	Include a **one-line explanation** of the map, e.g.: "This diagram shows where the refresh flow can fail under concurrency."	Helps non-expert judges understand the artifact's value. [1] [3]
1:55–2:00	Impact line about fewer bugs, safer reviews.	Optionally add: "Built for the Build with AI: Basics hackathon." if the rules ask to mention the challenge.	Ensures you meet any explicit video requirements. [5] [6]

Overall: your structure and messaging are correct; these tweaks just improve clarity, pacing, and rubric alignment. [2] [3] [4]

2. Tech stack: corrections and clarifications

Your stack table is good. A few precise refinements:

Layer	Your choice	Correction / clarification
Frontend	Next.js / Vite + Tailwind, dark terminal aesthetic, `/demo` route.	✅ Correct. Also ensure the `/demo` route: - Pre-loads the exact diff, answers, and results you'll show in the video. - Has a "Reset" button so you can re-run the demo cleanly if you mess up while recording.
Backend	Node.js / TS, `POST /api/analyze-diff` returning `{ questions, score, riskFlags, testSnippet, cognitiveMap }`.	✅ Correct. Add: - A **mock mode flag** (e.g., `?mock=true`) that bypasses the LLM and returns hardcoded demo data. - Simple logging (console or file) of requests for debugging during development.
Diff parsing	`parse-diff` (npm) instead of Git CLI.	✅ Correct. Just ensure you handle: - Multiple files in one diff (even if your demo uses one). - Basic metadata: filename, hunks, line numbers, so you can reference "Line 52" reliably.
AI engine	Gemini API + fallback/mock mode.	✅ Correct. In `spec.md`, briefly describe: - Prompt structure for question generation. - How you map answers to a score (even if it's a simple rule-based heuristic for the PoC).

One more practical tip: deploy the frontend + API early (e.g., Vercel/Netlify for Next.js) so your demo video uses a **live URL**, not just `localhost`. Judges often click through submissions. [4] [2]

3. Planning artifacts: what to add

Your plan for `scope.md`, `prd.md`, and `spec.md` is correct and matches the hackathon's emphasis on planning docs plus working software.[7] [8]

To maximize their impact:

scope.md

Include:

- **Goal:** "Prove that a reverse-interview + targeted test flow can surface cognitive debt in AI-generated code."
- **In scope:**
  - Single Git diff input.
  - 3-question reverse interview.
  - Simple scoring + risk flag.
  - One adversarial test snippet.
  - `COGNITIVE_MAP.md` generation.
- **Out of scope (explicitly):**
  - Full CI/CD integration.
  - Multi-file, monorepo analysis.
  - Enterprise SSO, permissions, etc.

This shows disciplined scoping, which judges like.[8] [9]

prd.md

Include:

- **Personas:**
  - "AI-assisted developer" (uses Cursor/Copilot daily).
  - "Tech lead / reviewer" (worried about unreviewable AI code).
- **Key user stories**, e.g.:
  - "As a reviewer, I want to know which parts of an AI-generated diff the author doesn't understand, so I can focus my review."
  - "As a developer, I want targeted tests for my weak spots, so I can validate my changes before merging."
- **Success metrics (for the PoC):**
  - Demo completes in ≤ 2 minutes.
  - Tool consistently flags the intended function as high-risk in demo data.
  - Generated artifacts (map, verdict, test) are understandable in < 30 seconds.

This ties directly to “Impact” and “Design” criteria. [3] [10] [11]

spec.md

Include:

- **Architecture diagram** (even a simple one): frontend → `/api/analyze-diff` → diff parser → LLM → response.
- **API contract:**
  - Request: `{ diff: string }`
  - Response: `{ questions[], score, riskFlags[], testSnippet, cognitiveMap }`
- **Scoring rubric (simple):**
  - Example: 0–1 vague/incorrect answers → High Risk; 2 → Medium; 3 → Low.
- **Test generation approach:**
  - Describe at a high level how you choose inputs (e.g., concurrency, 429 responses, nulls) based on the flagged function.

You don’t need a perfect algorithm; you need a **clear, plausible design** judges can follow. [12] [3]

4. Strategic next steps: minor corrections

Your step table is strong. Just clarify priorities and add one missing piece.

Step	Your action	Small correction / addition	Priority
1	Initialize `scope.md`, `prd.md`, `spec.md`.	✅ Do this first; keep each file concise (1–2 pages max).	Immediate
2	Prepare sample demo data (40-line diff, vague/good answers, test).	✅ Also create a **JSON fixture** your `/demo` route can load directly.	Day 1
3	Pitch at Cup of Joe Office Hours with 1-minute elevator pitch.	✅ Prepare 2 specific questions for Joe (about scope and emphasis: map vs tests).	Today 3 PM ET
4	Build Next.js/Vite prototype with `/demo` quick-loader.	✅ Add **mock mode** and deploy early to a public URL.	Day 1–2
5 *(add)*	Draft Devpost submission text (pitch, what it does, how you built it).	Do this **before** recording the final video so your script and text align.	Day 2
6 *(add)*	Record and edit the 2-minute demo video; upload to YouTube/Vimeo as public/unlisted.	Leave buffer time before the deadline for rendering and last-minute tweaks.	Day 2–3

These additions ensure your video, repo, and submission page tell one consistent story, which is exactly what judges reward. [2] [3] [4]

Verdict

Yes—your plan is now correctly refined:

- Demo script: well-structured, focused, and aligned with judging criteria.
- Tech stack: appropriate and pragmatic, with good risk mitigation (mock mode, `/demo` route).
- Planning docs: exactly the kind of artifacts this hackathon expects.
- Next steps: realistic and ordered well; with the two extra steps above, you'll be in a very strong position.

If you'd like, I can next:

- Draft a concrete outline for `scope.md` / `prd.md` / `spec.md` tailored to UnderstandGuard, or
- Turn your 2-minute table into a word-for-word voiceover script you can rehearse.

⁂

1. https://help.devpost.com/article/84-video-making-best-practices
2. https://info.devpost.com/blog/6-tips-for-making-a-hackathon-demo-video
3. https://info.devpost.com/blog/how-to-present-a-successful-hackathon-demo
4. https://webmcp.devpost.com/updates/46161-2-days-left-and-what-judges-actually-look-for
5. https://help.devpost.com/article/126-know-your-submission-steps
6. https://pe-hackathon.devpost.com/
7. hackideas.md
8. https://devpost.com/software/example-template-submission
9. https://inspire-hackathon.devpost.com/rules
10. https://impacthack26.devpost.com/rules
11. https://stempower-her-hacks.devpost.com/?ref_content=default&ref_feature=challenge&ref_medium=portfolio
12. https://furthering-the-future.devpost.com/rules